BAIL security

Token Mill V2
EVM

FINAL REPORT

May '2025

Disclaimer:

Security assessment projects are time-boxed and often reliant on information that may be provided by a client, its affiliates, or its partners. As a result, the findings documented in this report should not be considered a comprehensive list of security issues, flaws, or defects in the target system or codebase.

The content of this assessment is not an investment. The information provided in this report is for general informational purposes only and is not intended as investment, legal, financial, regulatory, or tax advice. The report is based on a limited review of the materials and documentation provided at the time of the audit, and the audit results may not be complete or identify all possible vulnerabilities or issues. The audit is provided on an "as-is," "where-is," and "as-available" basis, and the use of blockchain technology is subject to unknown risks and flaws.

The audit does not constitute an endorsement of any particular project or team, and we make no warranties, expressed or implied, regarding the accuracy, reliability, completeness, or availability of the report, its content, or any associated services or products. We disclaim all warranties, including the implied warranties of merchantability, fitness for a particular purpose, and non-infringement.

We assume no responsibility for any product or service advertised or offered by a third party through the report, any open-source or third-party software, code, libraries, materials, or information linked to, called by, referenced by, or accessible through the report, its content, and the related services and products. We will not be liable for any loss or damages incurred as a result of the use or reliance on the audit report or the smart contract.

The contract owner is responsible for making their own decisions based on the audit report and should seek additional professional advice if needed. The audit firm or individual assumes no liability for any loss or damages incurred as a result of the use or reliance on the audit report or the smart contract. The contract owner agrees to indemnify and hold harmless the audit firm or individual from any and all claims, damages, expenses, or liabilities arising from the use or reliance on the audit report or the smart contract.

By engaging in a smart contract audit, the contract owner acknowledges and agrees to the terms of this disclaimer.

1. Project Details

Important:
Please ensure that the deployed contract matches the source-code of the last commit hash.

Project	Audit Token Mill
Website	tokenmill.xyz
Language	Solidity
Methods	Manual Analysis
Github repository	https://github.com/traderjoe-xyz/token-mill-v2/tree/b5a9b83322658302b3ade065f7af9b19d0d69684/src
Resolution 1	https://github.com/traderjoe-xyz/token-mill-v2/tree/c2b2a07415e0596b1fe9b9d69b8b391d0513b60d/src

2. Detection Overview

Severity	Found	Resolved	Partially Resolved	Acknowledged (no change made)	Failed Resolution
High					
Medium	2	2			
Low	6	1	2	3	
Informational	7	1		6	
Governance	1			1	
Total	16	4	2	10	

2.1 Detection Definitions

Severity	Description
High	The problem poses a significant threat to the confidentiality of a considerable number of users' sensitive data. It also has the potential to cause severe damage to the client's reputation or result in substantial financial losses for both the client and the affected users.
Medium	While medium-level vulnerabilities may not be easy to exploit, they can still have a major impact on the execution of a smart contract. For instance, they may allow public access to critical functions, which could lead to serious consequences.
Low	Poses a very low-level risk to the project or users. Nevertheless, the issue should be fixed immediately.
Informational	Effects are small and do not pose an immediate danger to the project or users.
Governance	Governance privileges which can directly result in a loss of funds or other potential undesired behavior.

3. Detection

/Core

TMMarket

The TMMarket contract is deployed directly as an implementation contract with immutable parameters for specific trading configurations, these immutable parameters are:

- factory
- quoteToken
- liquidityA
- liquidityB
- sqrtRatioAX96 (lower price limit)
- sqrtRatioBX96 (mid-price limit)
- maxSupply

The contract then allows for buying and selling tokens via a UniV3-style mechanism by exposing two ranges.

Appendix: Swapping Paths (UV3)

The swap function can be invoked in four different ways whereas there are a total of 8 different call-paths whether a swap can be done within a range or crosses a range.

The following call-paths are possible:

1. Exact Input; X -> Y; within range
2. Exact Input; Y -> X; within range
3. Exact Output; X -> Y; within range
4. Exact Output; Y -> X; within range
5. Exact Input; X -> Y; outside range
6. Exact Input; Y -> X; outside range
7. Exact Output; X -> Y; outside range
8. Exact Output; Y -> X; outside range

In-depth explanation of all flows based on the UV3 implementation:

Swap within the range

exactIn

1. Swap function is invoked with the input amount and limitSqrtPrice

2. The swap step is calculated with the following necessary parameters:
    a. currentSqrtPrice
    b. targetSqrtPrice OR nextSqrtPrice (which is better for user)
    c. liquidity
    d. inputAmount (positive)
    e. fee (in BPS)

3. calculate amountRemainingLessFee
    a. amountRemaining * (1e6-fee) * 1e6

4. **calculate how much the range can "absorb" (round up)**
    a. X -> Y (getAmount0Delta)
        i. amountIn = (liquidityX96 * (priceCurrent - priceNext) / priceCurrent / priceNext
    b. Y -> X (getAmount1Delta)
        i. amountIn = liquidity * (nextPrice - currentPrice) / X96

5. amountRemainingLessFee can be fully absorbed

6. **Calculate new price**
    a. X -> Y (getNextSqrtPriceFromAmount0RoundingUp)
        i. nextPrice = (liquidityX96 * currentPriceSqrtX96) / (liquidityX96 + (amount*currentPriceSqrtX96))
    b. Y -> X (getNextSqrtPriceFromAmount1RoundingDown)
        i. nextPrice = currentPriceSqrtX96 + (amountX96 / liquidity)

7. **Calculate INPUT / OUTPUT**
    a. X -> Y
        i. amountIn (getAmount0Delta) = (liquidityX96 * (priceCurrent - priceNext) / priceCurrent / priceNext
        ii. amountOut (getAmount1Delta) = liquidity * (currentPrice - nextPrice) / X96
    b. Y -> X
        i. amountIn (getAmount1Delta) = liquidity * (nextPrice - currentPrice) / X96

ii. amountOut (getAmount0Delta) = (liquidity * X96 * (priceNext - priceCurrent) / priceNext / priceCurrent

exactOut

1. Swap function is invoked with the input amount and limitSqrtPrice

2. The swap step is calculated with the following necessary parameters:
   a. currentSqrtPrice
   b. targetSqrtPrice OR nextSqrtPrice (which is better for user)
   c. liquidity
   d. inputAmount (negative)
   e. fee (in BPS)

3. calculate amountRemainingLessFee
   a. amountRemaining * (1e6-fee) * 1e6

4. **calculate how much the range can provide as output amount (round down)**
   a. X -> Y (getAmount1Delta)
      i. amountOut = liquidity * (currentPrice - nextPrice) / X96
   b. Y -> X (getAmount0Delta)
      i. (liquidityX96* (priceNext - priceCurrent) / priceNext / priceCurrent

5. amountRemainingLessFee can be fully absorbed

6. **Calculate new price**
   a. X -> Y (getNextSqrtPriceFromAmount1RoundingDown)
      i. nextPrice = currentPriceSqrtX96 - (amountX96 / liquidity)
   b. Y -> X (getNextSqrtPriceFromAmount0RoundingUp)
      i. nextPrice = (liquidityX96) * currentPriceSqrtX96 / (liquidityX96 - (amount * currentPriceSqrtX96))

7. **Calculate INPUT / OUTPUT**
   a. X -> Y
      i. amountIn (getAmount0Delta) = (liquidityX96 * (priceCurrent - priceNext) / priceCurrent / priceNext
      ii. amountOut (getAmount1Delta) = liquidity * (currentPrice - nextPrice) / X96

b. Y -> X

i. amountIn (getAmount1Delta) = liquidity * (nextPrice - currentPrice) / X96

amountOut (getAmount0Delta) = (liquidityX96 * (priceNext - priceCurrent) / priceNext / priceCurrent

Swap outside the range

exactIn

1. Swap function is invoked with the input amount and limitSqrtPrice

2. The swap step is calculated with the following necessary parameters:
   a. currentSqrtPrice
   b. targetSqrtPrice OR nextSqrtPrice (which is better for user)
   c. liquidity
   d. inputAmount
   e. fee (in BPS)

3. calculate amountRemainingLessFee
   a. amountRemaining * (1e6-fee) * 1e6

4. **calculate how much the range can "absorb" (round up)**
   a. X -> Y (getAmount0Delta)
      i. amountIn = (liquidityX96 * (priceCurrent - priceNext) / priceCurrent / priceNext
   b. Y -> X (getAmount1Delta)
      i. amountIn = liquidity * (nextPrice - currentPrice) / X96

5. Range cannot absorb provided amount

6. Next price will be range boundary

7. X -> Y
   a. amountIn = amountIn from step 4
   b. amountOut (getAmount1Delta) = liquidity * (currentPrice - nextPrice) / X96

8. Y -> X
   a. amountIn = amountIn from step 4

b. amountOut (getAmount0Delta) = (liquidityX96 * (priceNext - priceCurrent) / priceNext / priceCurrent

exactOut

1. Swap function is invoked with the input amount and limitSqrtPrice
2. The swap step is calculated with the following necessary parameters:
   a. currentSqrtPrice
   b. targetSqrtPrice OR nextSqrtPrice (which is better for user)
   c. liquidity
   d. inputAmount
   e. fee (in BPS)

3. calculate amountRemainingLessFee
   a. amountRemaining * (1e6-fee) * 1e6

4. **calculate how much the range can provide as output amount (round down)**
   a. X -> Y (getAmount1Delta)
      i. amountOut = liquidity * (currentPrice - nextPrice) / X96
   b. Y -> X (getAmount0Delta)
      i. (liquidityX96 * (priceNext - priceCurrent) / priceNext / priceCurrent

5. Range cannot absorb provided amount

6. Next price will be range boundary

7. **X -> Y**
   a. amountIn (getAmount0Delta) = (liquidityX96 * (priceCurrentX96 - priceNextX96) / priceCurrentX96 / priceNextX96
   b. amountOut = outputAmount from step 4

8. **Y -> X**
   a. amountIn (getAmount1Delta) = liquidity * (nextPrice - currentPrice) / (2**128)
   b. amountOut = outputAmount from step 4

Core Invariants:

INV 1: During deployment, only token0 is being added as liquidity

INV 2: Both liquidites must be immutable

INV 3: reserve0 and reserve1 must always reflect the real reserves after a swap

INV 4: _sqrtRatio must be changed after each swap

INV 5: Fee must always be taken in tokenY

INV 6: During X to Y swap, fee is accumulated in X and swapped to Y

INV 7: During Y to X swap, fee is accumulated in Y and transferred out

INV 8: During X -> Y (swap down), currentPrice must be > limitPrice AND limitPrice >= A/B

INV 9: During Y -> X (swap up), currentPrice must be < limitPrice and limitPrice <= A/B

INV 10: After X -> Y (swap down), price must decrease

INV 11: After Y -> X (swap up), price must increase

INV 12: During _getDeltaAmounts after the first swap, deltaAmount must reflect how much is left to provide (exactIn) or how much is left to take out (exactOut)

INV 13: TMMarket implementation must be deployed by governance for each quoteToken

INV 14: A swap of 100 tokenX input and 95 + 5tokenX input must result in the same tokenY output amount

INV 15: Within X > Y swaps, fee must be swapped from X > Y. In case of 100 input and 5% fee, the newPrice not be different to swapping 100 input and 0% fee

INV 16: tokenY (fee) transfer out must have no impact on protocol solvency or swap logic or

INV 17: fee must be transferred to the factory

INV 18: _sqrtRatio must never go below sqrtRatioAX96

INV 19: _sqrtRatio must never go above sqrtRatioMaxX96

INV 20: During X > Y swap, balanceOf check must include inputAmount + fee in tokenX

INV 21: During Y > X swap, balanceOf must include inputAmount + fee in tokenY

INV 22: within exactOutput X>Y swap, the determined sqrtNextX96 must never be LOWER than sqrtTargetRatioX96

INV 23: within exactOutput Y>X swap, the determined sqrtNextX96 must never be LARGER than sqrtTargetRatioX96

Privileged Functions

- none

Issue_01	Excessive limitation due to uint128 usage for reserve1
Severity	**Low**
Description	The contract uses reserve0 to track tokenX amount in the contract which is the TMToken and reserve1 to track tokenY amount which is the quote token in the contract. The maxSupply for tokenX is limited to be uint127.max which means it will never be possible for reserve0 to overflow. However, under specific circumstances for example if there is a lot of liquidity in a range where the price is high that can result in the necessity to provide a large amount of tokenY in an effort to buy out the whole supply of tokenX or even just a part of the supply. This will never work because reserve1 can be a maximum of uint128.max. It will simply revert upon the increase attempt: *reserve1 = Math.addDelta128(reserve1, amount1);* Here is a simplified example: range = [100000; max] (nominal prices) supply = uint90.max > [250541448375047931186413801569 60, 340282366920938463463374607431768211455] > tokenX = 1237940039285380274899124223 > liquidity = 391471041909052129818716479427 > tokenY needed to buy out full liquidity = 1681355198536411260747040244364818836974 Furthermore, this can be amplified if there is a deviation in the decimals between tokenX and tokenY in favor of the price which then further increases the price of the pair. For example if tokenY is a token with 21 decimals.

Recommendations	Consider setting a reasonable limit for sqrtRatioMaxX96
Comments / Resolution	Partially resolved, the max price has been decreases to 2**1271-1.

Issue_02	Creator can snipe in same TX as market deployment
Severity	**Low**
Description	Currently there is no cooldown period between market creation and the first swap call which means a creator can buy tokens in the same transaction as the market creation, granting them self an unfair advantage.
Recommendations	Consider implementing a cooldown period after market creation from one block.
Comments / Resolution	Acknowledged

Issue_03	Application of uint128.max as upper range limit is inefficient
Severity	**Low**
Description	The contract determines the second range between sqrtRatioBX96 and uint128.max which means the distribution will have a very wide range and the liquidity depth will thus suffer. While this has the obvious side-effect of large slippage it is simply inefficient and not practicable for tokens to reach such a large price.
Recommendations	We recommend reconsidering the maximum allowed price.
Comments / Resolution	Partially resolved, the max price has been decreases to 2**1271-1.

Issue_04	Fee is taken against the favor of the protocol
Severity	**Low**
Description	In case of a X -> Y swap, the fee is provided as tokenX and *after* the end of the swap it is swapped to tokenY by using the post-swap price. This means a larger swap will accumulate more fees but it will also result in a price decrease due to the swap movement which then makes the fee “less valuable” because the fee is only being swapped at the end price. Contrary to that if one executes a Y -> X swap, the fee which is accrued is nominal which means it does not require a swap with an altered price.
Recommendations	Consider acknowledging this issue, as a simple solution for this issue seems to not exist.

Comments / Resolution	Acknowledged.

Issue_05	Redundant addition of feeAmount1
Severity	**Informational**
Description	Within the swap function, the amountIn return value from _getDeltaAmounts already includes the feeAmount. This is the reason why the balance check does not implicitly add the feeAmount on top (contrary to the UniswapV3 implementation). feeAmount1 is then defined, as it will be eventually transferred out to the factory. However, in the scenario of a Y -> X swap, the feeAmount1 is added towards reserve1 for the balance check while being zero and only being assigned at a later point in the code. This addition is basically redundant: *if (reserve1 + feeAmount1 > IERC20(token1).balanceOf(address(this))) revert InsufficientBalance1();*
Recommendations	Consider simply removing the addition.
Comments / Resolution	Resolved. feeAmount1 is now removed.

Issue_06	sqrtPriceX96 as slippage parameter is insufficient
Severity	**Informational**
Description	Whenever a swap is being executed a nominal fee in the form of the input amount is being taken from the swapper. This fee application does not fall under the slippage protection as the slippage protection is only based on the price. If for example the fee is 50% but the limit price is set very strict, the user would still lose a lot of tokens. Furthermore, depending on the liquidity depth, the output amount will be smaller/larger, which could for example be influenced by a swap before, which moves the range to the lower liquidity range which would then result in larger slippage and thus a lower output amount compared to the scenario where no previous swap has happened. This is of course all under the premise that the limit price will not be reached but it basically means that swaps can be ambiguous. This issue has only been rated as informational because it is expected that the router enforces such a minOutput.
Recommendations	Since the fee is immutable this does not seem to be a large risk, therefore we recommend simply acknowledging that issue and implementing a minOut slippage protection only on the router. We furthermore also recommend for the router contract to be audited because the protocol clearly makes use of such a contract.
Comments / Resolution	Acknowledged.

Issue_07	Transfer-tax tokens are not supported
Severity	**Informational**
Description	This contract is not compatible with transfer-tax tokens. If these token types are used for any purpose within the contract, this will result in down-stream issues and inherently breaks the accounting.
Recommendations	Consider not using these tokens.
Comments / Resolution	Acknowledged.

Issue_08	Read-only-reentrancy for various view functions
Severity	**Informational**
Description	The swap function optimistically transfers out the output tokens before reserves are updated which allows for read-only-reentrancy in various view functions in case tokenY is a token with a hook.
Recommendations	Consider acknowledging this issue.
Comments / Resolution	Acknowledged.

Issue_09	Safe casting in _getDeltaAmounts prevents from buying more than uint127.max
Severity	**Informational**
Description	Currently the _getDeltaAmounts function returns the following data: *return (nextSqrtRatioX96, Math.safeUint127(amountIn + feeAmountIn), Math.safeUint127(amountOut), feeAmountIn);* This may prevent purchasing OR providing a tokenY amount in one transaction which is larger than uint127.max, which is theoretically possible because only the tokenX supply is limited to uint127.max.
Recommendations	Consider acknowledging this issue.
Comments / Resolution	Acknowledged.

Issue_10	Potentially locked tokens
Severity	**Informational**
Description	Currently, it is expected that the inputAmount is transferred in before the swap function execution. In case where the inputAmount is larger than the required amount, these funds essentially remain locked in the contract. This however depends on the calculation logic in the router.
Recommendations	Consider acknowledging that issue. We do not recommend a skim call as this increases user flexibility and the likelihood of exploits.
Comments / Resolution	Acknowledged.

TMToken

The TMToken contract is a simple ERC20Upgradeable contract which is cloned by the TMFactory whenever a new market is being created. It is expected that a boilerplate implementation is already deployed which is then used as _tokenImplementation variable within the TMFactory contract.

Core Invariants:

INV 1: Mint must only be called by the TMMarket contract during the initialization

INV 2: Initialize must only be called by TMFactory during the market creation

INV 3: Initialize can only be called once

Privileged Functions

- none

No issues found

TMFactory

The TMFactory contract allows for the permissionless deployment of new markets for corresponding quote tokens. Ideally governance selects one or multiple quote tokens and sets a corresponding deployed TMMarket implementation for each specific quote token via the setMarketImplementation function.

Each implementation has their custom parameters:

- quoteToken: The token which is used to buy the TMToken
- sqrtRatioAX96: Starting price of first range
- sqrtRatioBX96: Starting price of second range
- maxSupply: Aggregate of amountX in first and second range
- liquidityA: Liquidity within first range
- liquidityB: Liquidity within second range

These custom parameters are immutable and will be the same for every deployed market tied to a specific quote token. Besides of that, there are multiple other parameters within the TMFactory contract which can be influenced by governance to impact the trading sentiment, such as:

- _minUpdateTime: Determines after what time the feeRecipient can be changed
- _protocolFeeShare: Determines how much of each swap fee goes towards the protocol as revenue
- _defaultFee: Determines the overall fee which is applied on each swap. This does not affect existing markets but will only be applied for new creations

On top of that, the TMFactory contract takes care of the fee accumulation for all participants from markets via the onFeeReceived function and allows claiming of fees via the collect function.

The contract leverages OZ's AccessControlUpgradeable RBAC module for governance interactions.

Appendix: Market Creation

The createMarket function allows for the permissionless market deployment with their corresponding quote token. There can be an arbitrary amount of markets for the same quote, deployed by the same or different addresses.

Each market has a non-unique name and symbol as well as a creator address which is tied to it. Whenever a new market is deployed, the MarketDetails struct which contains the following properties:

- initialized
- lastFeeRecipientUpdate
- creator
- feeRecipient

is pushed into the _details[market] mapping. Furthermore the market address is added to the _marketsByCreator[msg.sender] address set. The token and corresponding market are simply cloned using OZ's Clone library while the market is deterministically cloned, using the token address as salt which ensures the creation can never be DoS'd.

Core Invariants:

INV 1: Only a valid market can call onFeeReceived

INV 2: _tokenImplementation must point to the TMToken contract

INV 3: feeRecipient can only be changed if _minUpdateTime has passed since the last feeRecipient update

INV 4: _marketImplementation must point to the TMMarket contract for the corresponding quoteToken

INV 5: market creation must be permissionless

INV 6: collect is only allowed for own address OR for KTOM_FEE_RECIPIENT with KOTM_COLLECTOR_ROLE OR for PROTOCOL_FEE_RECIPIENT with PROTOCOL_FEE_COLLECTOR role

INV 7: updateMarkeDetails is only allowed for creator

INV 8: A market for a creator must always be marked within _details[market].creator AND _marketsByCreator[creator] set

Privileged Functions

- grantRole
- revokeRole
- setMinUpdateTime
- setProtocolFeeShare
- setDefaultFee
- setMarketImplementation
- setTokenImplementation

Issue_11	Governance Privilege: Stealing of fees
Severity	**Governance**
Description	Currently, governance of this contract has several privileges for invoking certain functions that can drastically alter the contracts behavior. Governance can set a malicious _marketImplementation contract which then calls the onFeeReceived function to inflate fees for an own address without actually transferring them in. Moreover, governance can set many different variables as mentioned within the privileged functions section.
Recommendations	Consider incorporating a Gnosis Multisignature contract as owner and ensuring that the Gnosis participants are trusted entities.
Comments / Resolution	Acknowledged.

Issue_12	Spoofing possibility for markets
Severity	**Medium**
Description	The creator address can be set to any address inside updateMarketDetails`allowing for spoofing. For example, someone creates a token and sets Vitalik's address as the owner. This trick some users. On top of that name and symbol can be spoofed to trick users and it can be immediately done *after* creation (in the same TX) and does not fall under _minUpdateTime check (if feeRecipient is not changed). This means if there are some rumours or announcements about a token creation from a big player with a publicly known address, a malicious user can simply do the following in the same TX: a) Create a new market with the apparent token name and symbol b) Snipe from own market in the same TX c) Set creator address to Vitalik's address in the same TX .. wait for users to purchase the token d) Dump on users
Recommendations	Consider not allowing to change the creator address to another address without explicit permission.
Comments / Resolution	Resolved, the creator transfer now incorporates the standard 2 step process where the new owner has to explicitly accept the transfer disallowing spoofing.

Issue_13	Creator can never accrue fees between [deployment; deployment + _minUpdateTime]
Severity	**Medium**
Description	Currently, it is not possible for the creator to accrue any fees right after the market deployment due to the limitation within the feeRecipient setting. It is simply not allowed to change the feeRecipient address until _minUpdateTime has passed. This is a significant limitation which is undesired and not intended per design.
Recommendations	Consider simply setting lastFeeRecipientUpdate to 0 during the market deployment OR allowing to pass an address as argument during the createMarket call.
Comments / Resolution	Resolved. Now feeRecipient is passed as a param during market creation.

Issue_14	No reasonable validation for protocolFeeShare
Severity	**Low**
Description	The setProtocolFeeShare function allows for setting the _protocolFeeShare . Currently, there is no validation which ensures that the setting will result in a reasonable value. The same applies to the _defaultFee value and _minUpdateTime.
Recommendations	Consider improving the validation to a reasonable value.
Comments / Resolution	Acknowledged.

/Libraries

SwapMath

The SwapMath contract is responsible for calculating input and output amount for a swap as well as the price after the swap. It furthermore calculates the corresponding fee for a swap. It is solely invoked by the TMMarket contract during a swap call as well as for view-only purposes.

Appendix: UV3 Math

The math is heavily inspired from Uniswap V3 and implements minor optimizations. The goal of the audit is **not to formally validate the Uniswap V3** math but to **compare the implemented math against the trusted implementation** and **uncover potential risks which stem from a deviation** between both implementations.

Below we will list all functions in TokenMillV2 and their UniswapV3 equivalent:

getLiquidityO <-> LiquidityAmounts.getLiquidityForAmountO

> (lowerX96 * upperX96) * amountO / (upperX96 - lowerX96) << X96

vs

> amountO * (upperX96* lowerX96 / X96) / upperX96 - lowerX96)

getAmountO <-> getAmountODelta

X -> Y (x is input amount)

> (liquidityX96 * (priceCurrentX96 - priceNextX96) / (priceNext96 * priceCurrentX96)

vs

> (liquidityX96 * (priceCurrentX96 - priceNextX96) / priceCurrentX96 / priceNextX96

Y -> X (x is output amount)

> liquidity * (priceNextX96 - priceCurrentX96) / (priceCurrentX96 * priceNextX96)

vs

> liquidity * (priceNextX96 - priceCurrentX96) / priceNextX96 / priceCurrentX96

getAmount1 <-> getAmount1Delta

X -> Y (Y is output amount)

> liquidity * (currentPriceX96 - nextPriceX96) / X96

vs

> liquidity * (currentPriceX96 - nextPriceX96) / X96

Y -> X (Y is input amount)

> liquidity * (nextPriceX96 - currentPriceX96) / X96

vs

> liquidity * (nextPriceX96 - currentPriceX96) / X96

getNextSqrtRatioFromAmount0 <-> getNextSqrtPriceFromAmount0RoundingUp

X -> Y (X is input amount)

> liquidityX96 * currentPriceSqrtX96 / (liquidityX96 + (amount * currentPriceSqrtX96))

vs

> (liquidityX96 * currentPriceSqrtX96) / (liquidityX96 + (amount*currentPriceSqrtX96))

Y -> X (X is output amount)

> liquidityX96 * currentPriceSqrtX96 / (liquidityX96 + (-amount * currentPriceSqrtX96))

vs

> (liquidityX96) * currentPriceX96 / (liquidityX96 - (amount * currentPriceX96)

getNextSqrtRatioFromAmount1 <-> getNextSqrtPriceFromAmount1RoundingDown

X -> Y (Y is output amount)

> ((currentPriceSqrtX96 * liquidity) + (-amountX96)) / liquidity

vs

> currentPriceX96 - (amountX96 / liquidity)

Y -> X (Y is input amount)

> ((currentPriceSqrtX96 * liquidity) + (amountX96)) / liquidity

vs

> currentPriceX96 + ((amountX96) / liquidity)

Core Invariants:

INV 1: getNextSqrtPriceFromAmount0 must always round up

INV 2: getAmount0/1 must round up when calculating input amount/ down when calculating output amount

INV 3: feeAmount must always be calculated based on the input amount

INV 4: within exactOutput X>Y swap, the determined sqrtNextX96 must never be LOWER than sqrtTargetRatioX96

INV 5: within exactOutput Y>X swap, the determined sqrtNextX96 must never be LARGERA than sqrtTargetRatioX96

INV 6: for exactInput swaps, nextPrice calculation rounds to not exceed the price. This price is used to calculate how much the user receives as output amount.

INV 7: for exactOutput swaps, nextPrice calculation rounds to exceed the price. This price is used to calculate how much the user must provide as input amount.

Privileged Functions

- none

Issue_15	Precision increase attempt can result in reverts
Severity	**Low**
Description	The getNextSqrtRatioFromAmount0/1 functions are derived from UniswapV3's getNextSqrtPriceFromAmount0RoundingUp and getNextSqrtPriceFromAmount1RoundingDown functions. One important deviation from the UniswapV3 functionality is the single lining arithmetic: *int256 denominator = int256(liquidity) + amount0 * int256(sqrtRatioX96);* UniswapV3 handles that different as it only multiplies amount * sqrtRatio: *if ((product = amount * sqrtPX96) / amount == sqrtPX96)* This deviation can result in reverts due to overflow of int256, for example: sqrtRatioX96 = type(uint128).max liquidity = 2199023255552 amount0 = 2 ** 127 - 1 Furthermore, an inherent limitation is existing because the operation is used for int256 instead of uint256. Therefore, in specific scenarios which are mostly related to a large price close to uint128, a revert can occur.
Recommendations	Consider if this exposes a bug in reality. If yes, consider setting the maximum price to smaller value.
Comments / Resolution	Resolved. Now *sqrtPX96* is limited to 2**127-1 which ensures that the formulas won't overflow

Issue_16	General UniV3 deviations
Severity	**Informational**
Description	The codebase is highly inspired by UniswapV3 but other than the above mentioned issues, there are further deviations in the codebase which however (during our assessment) did not expose any vulnerabilities. Nonetheless it is important to still mention these deviations in an effort to spark further inspiration for fuzz-testing. a) UniswapV3 has two dedicated functions for fetching the next tick within an active range: getNextSqrtPriceFromAmount0RoundingUp and getNextSqrtPriceFromAmount1RoundingDown. These functions have extra conditions for "add" which is based on whether to calculate something on input or output. Therefore, they always have the amount parameter as a uint256. This is a deviation from how it's handled in TokenMill, as the getNextSqrtPrice functions here are simplified to accept a int256 amount value which allows for code simplification. Beside the already mentioned issue above, this exposes another issue, as now the overflow check is not against a uint256 anymore but instead against a int256. Which is usually not considered a problem because UniswapV3 only works with uint128.max supply tokens. However, this property is not guaranteed for tokenY. This subtle change could also result in some rounding deviations, therefore we recommend adding additional fuzzing against the original UniswapV3 functions to ensure no deviations are happening. b) getLiquidity0 does not sort upper/lower in case it's incorrect, contrary to UniswapV3. It is however not an issue because it is only used in the constructor and therefore it is already explicitly validated. c) In UniswapV3 within the exactOutput scenario where the range provide the desired outputAmount, the amountOut is calculated

	based on the new price and then clamped to amountRemaining: *if (!exactIn && amountOut > uint256(-amountRemaining)) {* *amountOut = uint256(-amountRemaining);* *}* Within TokenMillV2, this is handled differently as the calculation step in that scenario is just skipped and the amountOut is immediately set to the remainingAmountOut. Logically this is correct as if the desired output amount can be honored, the contract should indeed pay out how much was requested. However, since UniswapV3 executes the calculation explicitly, this could result in some rounding deviations where the received amount is not actually what was requested but slightly higher. In that scenario, the full requested output amount would be paid out in TokenMillV2. We however could not find a way to exploit this case, as the sqrtNextX96 value is anyways round against the favor of the user, which means the user has to provide a rounded up inputAmount. d) During _getDeltaAmounts, the amountIn includes amountIn + fee which deviates from UniV3.
Recommendations	Consider acknowledging this issue and increasing test and fuzzing coverage among the raised concern areas.
Comments / Resolution	Acknowledged.